

Monte Carlo Estimation of π on HTCondor

Understanding Throughput Using a Standardized Workflow

Snikitha Siddavatam

August 04, 2026

Contents

Background	1
Project Description	2
Project Scope	2
Project Challenge	2
Project Vision	3
Project Deliverables	3
Project Outcome	3
Shortcomings and Limitations	3
Recommendations for Next Phase of Project	4
Lessons Learned	4
Project	4
Personal	4
Project Material Links and Descriptions	5

Background

High-throughput computing (HTC) systems such as HTCondor let researchers split a large workload into many small, independent jobs that run in parallel across a shared pool of machines. The promise of HTC is throughput: the more jobs a user can move through the system per hour, the faster their science gets done. In practice, however, throughput depends on factors that are hard for users to see or reason about: how many jobs are submitted at once, how resource requests (CPU, memory, disk) affect where jobs can

be matched, and how fair-share scheduling adjusts a user’s priority as they consume pool resources. Users routinely over-request memory “to be safe,” which shrinks the set of machines that can run their jobs and slows the whole workload down.

To study these effects systematically, we need a workload that is simple, embarrassingly parallel, and statistically well understood. Monte Carlo estimation of π fits perfectly: each job draws S uniform random points in the unit square and counts how many fall inside the quarter-circle ($x^2 + y^2 < 1$); the fraction converges to $\pi/4$, and the estimation error shrinks at the known rate of $1/\sqrt{N}$ in the total sample count N . Because every job is identical and independent, any variation in completion time reflects the behavior of the scheduling system itself rather than the workload.

Project Description

This project uses Monte Carlo estimation of π as a standardized benchmark workload for profiling job throughput on HTCondor. We submit J independent jobs, each generating a fixed $S = 10,000$ samples with a provably unique random seed derived from its cluster and process IDs, and accumulate results in chronological order of job completion. From each run we measure how the absolute error of the π estimate decreases as the cumulative sample count N grows, and how throughput and per-job turnaround behave as J scales across five orders of magnitude: 10, 100, 1,000, 10,000, and 100,000 jobs. A pipeline of aggregation and plotting scripts cross-references each job’s output file against the HTCondor log, keeps only verifiably completed jobs, and produces convergence, runtime, turnaround, and cross-scale milestone comparisons.

Project Scope

The focus of this project is throughput characterization under controlled conditions: all jobs within an experiment request identical CPU, memory, disk, and clock-time limits, so differences in completion time reflect scheduling and pool variation rather than workload differences. We concentrate on (1) scaling behavior as the job count J grows from 10 to 100,000, (2) reproducible completion-time staggering via submit-side assignment of per-job sleep durations from a single seed chain, and (3) how throughput changes as the `request_memory` value is varied from 500 MB to 64 GB for an otherwise identical workload. We are *not* focusing on improving the Monte Carlo estimator itself (its $1/\sqrt{N}$ convergence is a known baseline used for validation), on optimizing single-job performance, or on modifying pool configuration or scheduler internals; the pool is treated as the system under observation, not something we tune.

Project Challenge

The central challenge is making runs at different scales genuinely comparable. Throughput on a shared pool is not stationary: submitting a large number of jobs lowers the user’s fair-share priority relative to other users, so a 100,000-job run can experience different scheduling conditions than the 10-job run it is being compared against, and priority ideally needs to be reset between experiments. Correctness at scale is also nontrivial: with 100,000 jobs, a job must be counted as done only when both its output file exists at the Access Point *and* a matching `JOB_TERMINATED` entry appears in the HTCondor log, and the aggregation pipeline must order tens of thousands of completions by timestamp without double-counting or dropping jobs. Finally, reproducibility requires that nothing about the experiment is left to chance inside the jobs: sampling seeds must be provably unique across clusters, and the sleep durations that stagger completions must be regenerable from a single starting seed rather than chosen independently by each job.

Project Vision

The goal is a reusable, fully reproducible benchmark workflow that turns vague intuitions about HTC throughput into measured curves: how fast jobs move through the system at each scale, when each 10% completion milestone is reached, and how much throughput is lost when memory requests are inflated. The impact is practical guidance for pool users (evidence for how to size job counts and resource requests so that workloads finish sooner) and a standardized instrument that facilitators can rerun at any time to observe how the pool’s throughput behavior changes. Because the workload’s statistical behavior is known exactly, the same workflow doubles as an end-to-end validation harness: if the error curve does not fall at $1/\sqrt{N}$, something in the pipeline, not the pool, is wrong.

Project Deliverables

The project delivers a complete, rerunnable benchmark workflow built around a small set of scripts. The workload itself consists of `mc_pi.py`, which draws $S = 10,000$ samples per job using a seed derived uniquely from the HTCCondor cluster and process IDs, and `mc_pi.sub`, the submit description that fixes identical CPU, memory, disk, and clock-time requests for every job in an experiment. On the analysis side, `aggregate.py` cross-references each job’s output file against the HTCCondor event log so that only verifiably completed jobs (output present *and* a matching `JOB_TERMINATED` entry) are counted, and orders completions chronologically to build the cumulative sample count N . A family of plotting scripts (`make_graphs.py`, `runtime_graph.py`, `milestone_times.py`, `milestone_scatter.py`, and `make_memory_graphs.py`) turns the aggregated results into convergence, runtime, turnaround, milestone, and memory-sweep figures.

Beyond the scripts, the deliverables include the measured datasets and figures themselves: complete runs at $J = 10, 100, 1,000, 10,000$, and $100,000$ jobs with per-scale runtime, turnaround, and completion-scatter plots; cross-scale milestone comparisons recording when each 10% completion milestone was reached; and a memory sweep varying `request_memory` from 500 MB to 64 GB on an otherwise identical 1,000-job workload. A companion simulator (`memory_simulator.Rmd`) models how memory requests shrink the set of matchable machines, allowing the measured throughput losses to be compared against a simple analytical expectation. Links to all scripts and figures are provided at the end of this report.

Project Outcome

The project achieved its central goal: throughput on the pool is no longer a vague intuition but a set of measured curves. The runs confirm that the workflow scales cleanly across five orders of magnitude in job count, that the error of the π estimate falls at the expected $1/\sqrt{N}$ rate (validating the pipeline end to end), and that completion-time behavior at each scale can be summarized in reproducible milestone charts. The memory sweep quantifies the cost of over-requesting: inflating `request_memory` for an identical workload measurably delays completion by shrinking the set of machines a job can match, giving concrete evidence for the standard facilitation advice to request only what a job actually needs.

The practical impact is twofold. For pool users, the results translate directly into guidance on sizing job counts and resource requests so that workloads finish sooner. For facilitators, the workflow is a standardized instrument: because every job is identical and statistically well understood, it can be rerun at any time to observe how the pool’s throughput behavior has changed, with the known convergence rate serving as a built-in correctness check.

Shortcomings and Limitations

The largest caveat is that the pool is a shared, non-stationary system, and the experiments only partially control for that. Fair-share priority drifts as large runs consume resources, so a 100,000-job run experiences

different scheduling conditions than the 10-job run it is compared against; without resetting priority between experiments (which requires administrator action), cross-scale comparisons carry some confounding from the user’s own submission history. Each configuration was also measured over a limited number of runs at particular times of day, so the curves capture the pool’s behavior during those windows rather than a long-run average across varying background load.

The results are likewise specific to this workload and this pool. Jobs here are short, CPU-light, and transfer almost no data, so the conclusions say little about workloads dominated by long runtimes, large input transfers, or GPU requests, and the measured memory-request penalties depend on this pool’s particular mix of machine sizes. Finally, the workflow observes the scheduler from the outside: it can measure *that* throughput changes with scale and memory requests, but it cannot attribute those changes to specific scheduler decisions without access to pool-side logs, which was outside the scope of this project.

Recommendations for Next Phase of Project

The most valuable next step is to turn the workflow from a one-time experiment into a recurring instrument. Because every run is identical and self-validating, the center could schedule the benchmark to run automatically (for example, weekly at a fixed job count and memory request) and archive the milestone curves over time, building a longitudinal picture of pool throughput that no single measurement window can provide. Pairing those recurring runs with a fair-share priority reset, coordinated with pool administrators, would remove the largest confound identified in this project and make cross-scale comparisons genuinely apples-to-apples.

With another four weeks as a Fellow, I would extend the sweep dimensions beyond memory: varying `request_cpus` and `request_disk` the same way `request_memory` was varied would complete the picture of how each resource request shapes matchability, and adding a controlled input-transfer payload would test whether the conclusions hold for data-heavier workloads. I would also close the attribution gap by requesting access to pool-side scheduler logs, so that observed throughput changes could be tied to specific matching and preemption decisions rather than inferred from the outside.

Lessons Learned

Project

The deliverables and their limitations point to a single underlying truth: on a shared pool, the workload is the easy part and the *system* is the experiment. A statistically trivial job like Monte Carlo π becomes scientifically useful precisely because it is boring: with the estimator’s $1/\sqrt{N}$ behavior known exactly, every surprise in the data can be attributed to the scheduler, the pool, or the pipeline rather than the workload. Any Fellow doing this project would also learn that correctness at scale is a bookkeeping problem before it is a statistics problem: at 100,000 jobs, deciding what counts as a “completed job” (output file present *and* a matching log entry) matters more than any downstream analysis.

The second broad lesson is that reproducibility on an HTC system must be designed in from the submit side. Seeds, sleep durations, and resource requests all had to be fixed or derivable before any job started, because anything left to per-job chance becomes unrecoverable noise across tens of thousands of jobs. Conversely, some things cannot be controlled from the user side at all, fair-share priority drift chief among them, and recognizing the boundary between what an experiment can and cannot hold fixed is itself a result.

Personal

Executing this project gave me working fluency with the full HTCCondor lifecycle: writing submit descriptions, deriving unique seeds from cluster and process IDs, and, most usefully, parsing the HTCCondor event log to

cross-verify completions, a technique I had not needed before and now consider essential for any large submission. Debugging the aggregation pipeline at the 100,000-job scale taught me to distrust file counts alone and to build verification into data collection rather than bolting it on afterward.

On the analysis side, I sharpened a practical workflow for scaling studies: structuring per-scale runs into parallel directory trees (`mc_runs_10` through `mc_runs_100000`), writing plotting scripts that operate uniformly across scales, and using R Markdown to keep the analytical model (the memory simulator) and the measured results in documents that can be regenerated from scratch. Building the simulator alongside the measurements also changed how I approach experiments generally: having a simple analytical expectation to compare against made it far easier to tell when a measured effect was real and when it was an artifact of the pipeline.

Project Material Links and Descriptions

`snikithasiddavatam`

- My folder in the Fellows GitHub repository, containing all scripts, submit files, per-scale run directories, generated figures, and this report.